
ETAT DE L'ART

Résumé de lectures et positionnement

Ngnapa Ashley

Contents

1	Explication des mecanismes d'isolation	3
1.1	La virtualisation	3
1.2	le sandboxing	3
1.3	la conteneurisation ou virtualisation niveau système d'exploitation	3
1.4	Etude comparative des trois technologies	3
2	contexte et enjeux de notre travail	3
3	probleme et question de recherche	3
4	La revue de litterature	3
4.1	Volunteer grid computing vs volunteer cloud computing vs mobile volunteer computing	3
4.2	Tableau recapitulatif des systemes de volontariat existants	3
4.3	Tableau recapitulatif des mecanismes d'isolation existants	6
4.4	Tableau récapitulatif des points clés des articles lus	8
5	Positionnement final face aux articles lus	13
6	présentation d'une architecture minimaliste de notre solution	13

1 Explication des mecanismes d'isolation

1.1 La virtualisation

Le partage de ressources existe aujourd'hui grace a une multitudes de technologies qui ont su exploiter la possibilité d'isoler les ressources d'un hôte ainsi que leur utilisation. L'un des mecanismes phares est la virtualisation. La virtualisation, dans sa forme la plus simple, est un mécanisme permettant d'abstraire le matériel et les ressources système d'un système d'exploitation donné. [1, p.1]. C'est une abstraction complete du matériel physique de l'ordinateur créant des environnements d'exécution totalement isolés appelés machines virtuelles. L'hyperviseur est donc la couche d'abstraction qui existe entre le système d'exploitation et les machines virtuelles, qui effectue le travail de virtualisation des ressources matérielles. Il existe plusieurs types de virtualisation:

- La virtualisation complète: dite de type 1 ou bare metal, où l'hyperviseur est directement sur le matériel. C'est le cas de VMware, Xen, KVM...
- La virtualisation hébergée ou de type 2: Ici, l'hyperviseur est au-dessus d'un OS hôte. C'est le cas de VirtualBox, VMware Workstation...
- la paravirtualisation: C'est une technique de virtualisation où le système d'exploitation invité est conscient d'être virtualisé et est modifié pour communiquer directement avec l'hyperviseur via des « hypercalls »

Ce mecanisme de cloisonnement des ressources de l'ordinateur fournit une isolation forte au niveau du matériel (notamment CPU, RAM, Stockage, réseau), au niveau du noyau car chaque VM a son propre OS, et la gestion des ressources se fait par l'hyperviseur. Mais indépendamment de l'architecture, un système d'exploitation complet est toujours encapsulé dans la machine virtuelle. Cela signifie que l'image de la machine virtuelle sera très volumineuse, et que la simulation du périphérique matériel virtuel entraînera également un surcoût de performances plus important. [2, p.3]

1.2 le sandboxing

1.3 la conteneurisation ou virtualisation niveau système d'exploitation

1.4 Etude comparative des trois technologies

2 contexte et enjeux de notre travail

Les ressources existantes ont naturellement été installées et conçues pour des usages autres que le calcul distribué (par exemple, traitement de texte de bureau, accès à l'information sur le web, feuilles de calcul, etc.). Ces ressources doivent donc être exploitées sans perturber leur utilisation principale. Et pour atteindre un haut degré d'utilité, le calcul distribué doit prendre en charge un grand nombre d'applications de valeur – il doit être facile de déployer des applications sur la plateforme – et garantir la sécurité de l'application et de ses données pendant son exécution sur le réseau.

3 probleme et question de recherche

4 La revue de litterature

4.1 Volunteer grid computing vs volunteer cloud computing vs mobile volunteer computing

4.2 Tableau recapitulatif des systemes de volontariat existants

Système	Statut	Période	Isolation (Type + technologie)	Overhead (%)	Contrôle dynamique ressources	Multi-plateforme (Liste OS)
BOINC (Berkeley Open Infrastructure for Network Computing)	Actif	1999 - présent	Sandbox OS-natif. Linux: user/group isolation. Windows: separate account. VirtualBox pour VM optionnelle.	5-15% (natif) 20-30% (VM)	Oui (ajustement CPU, GPU, RAM, réseau via préférences)	Windows, Linux, macOS, Android, FreeBSD
SETI@home (Search for Extraterrestrial Intelligence)	Hibernation (2020)	1999 - 2020	Basé sur BOINC. Isolation via sandbox BOINC.	5-15% (hérité de BOINC)	Oui (via BOINC)	Windows, Linux, macOS (via BOINC)
XtremWeb	Abandonné (remplacé par XtremWeb-HEP)	2000 - 2012	Sandbox Java. Isolation processus basique.	10-20% (JVM overhead)	Limité	Windows, Linux, macOS (via JVM)
XtremWeb-HEP	Actif (développement limité)	2012 - présent	Sandbox Java + Docker optionnel. User isolation.	10-25% (Java ou conteneur)	Limité (configurations statiques)	Windows, Linux, macOS
HTCondor (High Throughput Computing)	Actif	1988 - présent	Isolation processus OS. Support Docker, Singularity. User-based isolation.	5-10% (natif) 10-20% (conteneur)	Oui (ClassAds, dynamic slots, policies)	Windows, Linux, macOS, Unix
Folding@home	Actif	2000 - présent	Sandbox propriétaire. Isolation processus. GPU isolation via drivers.	5-15%	Oui (contrôle CPU, GPU via client)	Windows, Linux, macOS
World Community Grid	Actif	2004 - présent	Basé sur BOINC. Isolation héritée.	5-15% (BOINC)	Oui (via BOINC)	Windows, Linux, macOS, Android
Einstein@Home	Actif	2005 - présent	Basé sur BOINC. Isolation héritée.	5-15% (BOINC)	Oui (via BOINC)	Windows, Linux, macOS, Android
Rosetta@home	Actif	2005 - présent	Basé sur BOINC. Isolation héritée.	5-15% (BOINC)	Oui (via BOINC)	Windows, Linux, macOS
GPUGRID	Actif	2007 - présent	Basé sur BOINC. Isolation GPU via drivers.	5-20%	Oui (via BOINC + GPU control)	Windows, Linux (GPU requis)

Système	Statut	Période	Isolation (Type + technologie)	Overhead (%)	Contrôle dynamique ressources	Multi-plateforme (Liste OS)
Gridcoin	Actif	2013 - présent	Utilise projets BOINC existants. Pas d'isolation propre.	Variable (dépend du projet)	Non (délégué aux projets)	Windows, Linux, macOS (via BOINC)
Golem Network	Actif	2016 - présent	VM (Wasmtime, Qemu). Forte isolation.	20-40% (VM overhead)	Limité	Windows, Linux, macOS
iExec	Actif	2017 - présent	Docker containers. SGX enclaves optionnel.	10-25%	Oui (via orchestration Docker)	Linux principalement
Volpex	Projet recherche (inactif)	2018 - 2020	Containers légers. Namespaces Linux.	5-10%	Oui (cgroups dynamiques)	Linux uniquement
VC-UY	En développement	2024 - présent	Namespaces, cgroups, seccomp. Rust-based.	Cible: ~5%	Oui (ajustement temps réel via API)	Linux

4.3 Tableau recapitulatif des mecanismes d'isolation existants

Ce tableau présente une analyse comparative détaillée des différents systèmes de conteneurisation et de virtualisation pertinents pour le calcul volontaire. L'analyse est structurée autour de dix critères essentiels permettant d'évaluer chaque solution selon plusieurs dimensions techniques et pratiques.

Système	Isolation	Overhead	Contrôle ressources dynamique	Multi-plateforme	Sécurité	Onboarding	Avantages	Inconvénients	Manquements
Docker	Namespaces, cgroups, capabilities	Faible (2-5%)	Limité via API Docker	Linux, Windows (WSL2), macOS (VM)	AppArmor, SELinux, seccomp	Simple, documentation riche	Écosystème mature, large communauté, Hub Docker	Nécessite daemon root, overhead réseau	Pas de contrôle dynamique natif, isolation moins forte que VM
BOINC runtime environment	Account-based sandboxing, permissions Unix	Très faible (<1%)	Aucun (isolation statique)	Linux, Windows, macOS, Android	Comptes utilisateurs dédiés, permissions fichiers	Complexe pour développeurs, simple pour volontaires	Écosystème scientifique établi, nombreux projets	Isolation faible, vulnérable aux escalades de privilèges	Pas de portabilité des applications, dépendance plateforme
LXC	Namespaces, cgroups, capabilities, AppArmor/SELinux	Très faible (1-3%)	Oui (via lxc-cgroup, API)	Linux uniquement	AppArmor, SELinux, seccomp	Complexe, nécessite configuration système	Mature, base de Docker, communauté active	Configuration complexe, moins d'outils que Docker	Conteneurs système complets (pas applicatifs), gestion réseau complexe
Singularity	Namespaces, pas de daemon root	Faible (2-4%)	Limité	Linux, expérimental Windows/macOS	Exécution sans privilèges, intégration HPC	Orienté scientifique/HPC, courbe apprentissage moyenne	Adoption forte en HPC, images Docker compatibles	Pas de daemon, sécurité renforcée	Écosystème moins riche que Docker, focus HPC uniquement
Linux VServer	Isolation au niveau noyau (patché)	Très faible (<2%)	Limité	Linux (noyau patché requis)	Isolation réseau, quotas ressources	Très complexe, maintenance difficile	Projet legacy, peu actif	Nécessite patch noyau, communauté réduite	Technologie dépassée, non maintenue activement
Rocket (rkt)	Namespaces, cgroups, pas de daemon	Faible (2-4%)	Via systemd	Linux uniquement	SELinux, intégration systemd	Complexe, moins de documentation	Compatible OCI, sécurité renforcée	Projet archivé (2020), pas de daemon central	Plus maintenu, communauté disparue
REG (Restricted Execution Grid)	Sandboxing système, chroot	Faible (3-5%)	Non	Multi-plateforme	Environnement chroot, permissions	Orienté grid computing	Spécialisé pour grids	Isolation limitée, technologie ancienne	Documentation limitée, peu d'adoption
OpenVZ	Isolation noyau, namespaces modifiés	Très faible (1-2%)	Oui (vzctl)	Linux (noyau patché)	Isolation réseau, quotas	Complexe, nécessite noyau spécifique	Virtualization VPS établie	Nécessite patch noyau, moins flexible que LXC	Évolution vers Virtuozzo (commercial)
VoC-UYR (my final runtime)	Namespaces, cgroups, capabilities, seccomp	Très faible (<2%)	Oui (ajustement temps réel par volontaire)	Linux natif		Simple pour volontaires, API claire pour coordinateur	Spécialisé volunteer computing, intégration Django/MongoDB	Rust (sécurité mémoire), contrôle dynamique ressources	Projet nouveau, communauté à construire, dépendance WSL2 pour Windows

hline

4.4 Tableau récapitulatif des points clés des articles lus

Le tableau ci dessous constitue un outil d'analyse systématique de la littérature scientifique dans le domaine du calcul volontaire (volunteer computing). Il est organisé en sept colonnes qui permettent d'examiner chaque article de recherche selon une méthodologie structurée et cohérente. La première colonne identifie l'article étudié, suivie par une analyse du contexte et des enjeux qui motivent la recherche. La troisième colonne expose la question générale ou problématique centrale abordée par l'article.

Les colonnes suivantes approfondissent l'analyse critique : l'état de l'art présente les travaux existants et le contexte académique, tandis que la colonne "Problème/Insuffisances détectés" identifie les lacunes ou limites des approches actuelles. La sixième colonne décrit le modèle proposé par les auteurs et sa mise en œuvre concrète, c'est-à-dire la solution innovante apportée. Enfin, la dernière colonne synthétise l'apport spécifique de chaque article pour le projet de recherche personnel, permettant ainsi d'établir des liens entre la revue de littérature et les objectifs du chercheur. Cette structure facilite la comparaison entre différentes approches et l'identification des tendances et contributions majeures dans le domaine du calcul volontaire.

Article	Contexte et Enjeux	Question générale	État de l'art	Problème/Insuffisance détectés	Modèle proposé et Mise en œuvre	Apport pour mon sujet
Survey and taxonomy of volunteer computing	Croissance exponentielle des besoins en puissance de calcul pour les applications scientifiques. Emergence du calcul volontaire pour une meilleure exploitation des ressources peu utilisées de milliards d'appareils. Les enjeux principaux: • disponibilité intermittente • heterogeneité des ressources • Non intrusivité dans les activités du propriétaire de l'appareil • Sécurité et passage a l'échelle	Comment classifier et organiser les différents systèmes de calcul volontaire pour mieux comprendre leur fonctionnement et leurs applications?	Revue des plateformes existantes (BOINC, XtremWeb, HTCondor). Analyse des architectures distribuées. Historique depuis SETI@home (1999).	Hétérogénéité des ressources. Fiabilité et disponibilité variables. Manque de taxonomie unifiée pour comparer les systèmes. Sécurité et confidentialité des données.	Taxonomie basée sur 5 dimensions: architecture, gestion des ressources, modèle de sécurité, types d'applications.	Fournit un cadre conceptuel pour comprendre les différentes approches d'allocation de ressources; Propose des directives sur la gestion de la non intrusivité ; Souligne les benefices de l'utilisation des conteneurs ; Propose plusieurs autres systèmes de calculs volontaires ayant des architectures fonctionnelles, pouvant inspirer a la conception de la notre
An Effective Docker Image Slimming Approach Based on Source Code Data Dependency Analysis						

Article	Contexte et Enjeux	Question générale	État de l'art	Problème/Insuffisances détectés	Modèle proposé et Mise en œuvre	Apport pour mon sujet
An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment	L'essor de la technologie des conteneurs pour des virtualisations légères au niveau du système d'exploitation. Les enjeux principaux ici sont • L'instantiation et le démarrage rapide de conteneurs pour servir le besoin sans cesse croissant des utilisateurs • exploitation et maintenance a grande echelle des conteneurs	comment concevoir un conteneur ultra-léger capable de minimiser l'environnement d'exécution tout en maximisant le partage des ressources mémoire entre les instances ?	Evolution des systèmes basés sur l'hyperviseur, vers les conteneurs Linux utilisant les mécanismes de namespaces et de cgroups pour l'isolation; Exploitation de travaux connexes introduisant les concepts d'unikernel et de partage de memoire[2, p. 3-4]	Les inconvenients ou lacune de la solution présentée ici sont en plus des limitations inherentes aux technologies de conteneur: La dependance au réseau pour generation, l'absence d'isolation du réseau	Les technologies dominantes comme Docker présentent des lacunes importantes : elles conservent une forme de "système d'exploitation" qui encapsule des fonctions inutilisées, créant une redondance inutile dans le volume des images. Les auteurs proposent REG (Runtime Environment Generation), un moteur de gestion qui abandonne le concept d'image fixe au profit du code comme image. REG utilise principalement les liens physiques pour le partage des ressources lors de l'instanciation des conteneurs	Réduction de l'utilisation de la memoire et optimisation du temps de demarrage graces aux diverses methodes de partage de memoire présentées; et l'utilisation du sf overlays pour garantir l'intégrité des ressources et librairies partagées; Idée de remplacement des images par des templates préconcus et adaptés aux besoins; architecture client serveur, utilisation de namespaces et C groups
Bonic						
Characterizing and optimizing Kernel resource isolation for container						
Containerization and its Architectures A Study						

Article	Contexte et Enjeux	Question générale	État de l'art	Problème/Insuffisances détectés	Modèle proposé et Mise en œuvre	Apport pour mon sujet
High Performance Containers(2018)	L'enjeu ici est de fournir une virtualisation performante et légère capable de supporter des applications intensives en calcul (CPU) et intensives en données. (natif)	Est ce que l'architecture de RKT est meilleure pour l'exécution des tâches complexes dites HPC (high performance computing) par rapport a ses prédecesseurs Docker et LXC ?	Les auteurs dans leurs lectures ont constaté que La virtualisation au niveau du système d'exploitation (conteneurs) est préférée pour le HPC car elle partage le noyau de l'hôte, utilisant les namespaces et cgroups de Linux pour l'isolation, ce qui consomme moins de CPU et de mémoire que les machines virtuelles	Compleixté de la distribution des imaes, démarrage lent, stade d'évolution retrogradé	Les auteurs évaluent les performances de Rkt, Docker et LXC en utilisant les benchmarks HPL (Linpack) pour le calcul intensif et Graph500 pour les données intensives, sur des configurations à nœud unique et en cluster de conteneurs. L'implémentation met en évidence que l'architecture de Rkt, dépourvue de processus démon, permet de réduire la surcharge globale et d'améliorer le temps d'exécution.	L'article met en exergue le fait que l'absceance d'un demon central dans le runtime permet de maximiser les performances CPU en reduisant son utilisation par des changements de contexte et autres, tout en minimisant l'utilisation des ressources de l'hôte

Article	Contexte et Enjeux	Question générale	État de l'art	Problème/Insuffisance détectés	Modèle proposé et Mise en œuvre	Apport pour mon sujet
Linux Namespaces and cgroups as OS Primitives (2018)	Cet article s'inscrit dans le contexte du passage des machines virtuelles aux environnements d'isolations de services plus légers. L'enjeu majeur réside dans la recherche d'un équilibre entre une isolation forte et l'efficacité de l'utilisation des ressources	La question de recherche ici pourrait être comment les technologies de namespaces et Cgroups constituent le socle des systèmes de conteneurisation existants, et quelle est l'efficacité de leurs propriétés d'isolation par rapport aux approches de virtualisation conventionnelles par hyperviseur	Les auteurs présentent les résultats de leurs lectures portant principalement sur la différence qui réside entre la virtualisation complète, et la virtualisation niveau OS (conteneurisation). De plus, Les recherches antérieures confirment que les conteneurs atteignent des performances quasi natives grâce à l'usage direct du noyau	L'auteur analyse en détail sept types d'espaces de noms pour partitionner les ressources du noyau, et les cgroups pour surveiller et limiter le CPU, la mémoire et les E/S. Pour la mise en œuvre, l'article évalue les performances de Docker par rapport à KVM en réalisant des benchmarks pour mesurer la surcharge sur le CPU, la mémoire, le réseau et les disques.	L'insuffisance majeure identifiée est la faiblesse des garanties d'isolation par rapport aux VM basées sur un hyperviseur (augmentation de l'espace d'attaque, sécurité matérielle faible...)	Cet article nous permet de savoir que l'utilisation des technologies de namespaces et Cgroups pour notre runtime nous garantit un pourcentage maximal de surcharge de 5, explique clairement ces technologies sous linux pour une implémentation plus fluide.
Singularity						
A Performance Comparison of Container-Based Virtualization Systems for MapReduce Clusters (2014)						

- 5 Positionnement final face aux articles lus
- 6 présentation d'une architecture minimaliste de notre solution

References

- [1] “Analysis of virtualization technologies for high performance computing environments.”
- [2] “An ultra-lightweight container that maximizes memory sharing and minimizes the runtime environment.”